Uncertainty-Aware Multi-Agent Ransomware Detection via Decision MLP Fusion, Score-Space Augmentation, and Bayesian Triage Policy Optimisation

Henry Kabuye*, Biju Issac*, Senior Member, IEEE, Rahul Yumlembam*, Zahra Farzadpour*, and Jeyamohan Neera*, Member, IEEE

Abstract—Ransomware detection under operational constraints demands more than a binary classification verdict: it requires a principled mechanism for expressing per-sample uncertainty, fusing evidence from heterogeneous modalities, and converting uncertainty into actionable triage decisions that balance autonomous containment against analyst escalation. We present a heterogeneous multi-agent detection framework that addresses these requirements through four coupled contributions. First, three specialised agents—a Semantic Convolutional Neural Network (CNN) operating on static Portable Executable (PE) embeddings ($\mathbb{R}^{384}$), a Behavioural CNN operating on 16-dimensional runtime telemetry windows from a thin hypervisor ($\mathbb{R}^{16}$), and a Knowledge CNN operating on ATT&CK tactic embeddings ($\mathbb{R}^{d}$)—each employ Monte Carlo (MC) Dropout to produce a predictive probability $\hat{p}_i$ and an epistemic uncertainty σ_i per forward pass. Second, a Decision MLP replaces heuristic mean fusion: it learns a calibrated aggregate risk score from the six-dimensional score pool $(\hat{p}_i, \sigma_i)_{i=1}^{3}$, reducing Expected Calibration Error (ECE) from 0.094 to 0.018. Third, Conditional Tabular GAN (CTGAN) augments the score-space training distribution to address class imbalance in the fusion layer without retraining the base agents. Fourth, a triage policy defined over four thresholds $(\tau_{\text{high}}, \tau, \kappa, \kappa_{\text{low}})$ is optimised via Bayesian Tree-structured Parzen Estimation (TPE) over 300 trials using the objective $J(\tau, \kappa) = \text{bal_acc}(\tau, \kappa) - \lambda \cdot \text{esc_rate}(\tau, \kappa)$, reducing the escalation rate from 85% to 25% while increasing balanced accuracy from 0.525 to 0.866. A Peer Consultation mechanism prevents knowledge collapse under concurrent benign–ransomware execution, and an Expert Feedback Loop with Elastic Weight Consolidation (EWC) enables continual adaptation to novel families with negligible catastrophic forgetting (ΔAUPRC = 0.001 on retained families). Evaluation across four generalisation axes (random, temporal, family holdout, hardware holdout) over six independent seeds on RanSMAP, RDset, and RansKnow demonstrates that the framework achieves AUROC 0.996 on static PE detection, AUROC 0.963 on runtime behavioural detection, and AUROC 0.980 under temporal holdout, while a Mondrian conformal prediction wrapper provides distribution-free coverage guarantees with 77% of traffic auto-classified at 99% coverage.

Index Terms—Ransomware detection, multi-agent systems, Monte Carlo Dropout, uncertainty quantification, Decision MLP, score-space augmentation, CTGAN, Bayesian optimisation, triage policy, conformal prediction, continual learning, elastic weight consolidation, threat intelligence.

*Department of Computer and Information Sciences, Northumbria University, Newcastle, NE1 8ST, UK. E-mail: {kabuye-henry2017, bissac, rahul.yumlembam, zahra.farzadpour, jeyamohan.neera}@northumbria.ac.uk.

B. Issac is the corresponding author. E-mail: bissac@ieee.org.

Manuscript received 2026.

I. Introduction

Ransomware attacks impose escalating operational and financial costs on enterprises and critical infrastructure [1], [2]. State-of-the-art detectors—whether signature-based antivirus engines, endpoint detection and response (EDR) platforms, or machine-learning classifiers trained on file-level features—converge on a common architectural limitation: they produce a single scalar confidence score and map it to a binary verdict. When a novel variant or an obfuscated execution pattern degrades that score silently, no signal informs the operator that the verdict is unreliable.

We identify three structural gaps in the current state of practice:

1) No principled uncertainty. A softmax output from a deterministic neural network is a point estimate of class membership, not a probability over models. It cannot distinguish a confident correct prediction from a confident incorrect one.
2) Heuristic modality fusion. Combining static PE features with runtime telemetry via equal-weight averaging discards the per-modality reliability signal; a semantic agent operating on encrypted PE sections and a behavioural agent observing obfuscated I/O patterns should not carry equal weight.
3) Binary triage. An alert generated by every uncertain detection overwhelms analysts; equally, routing every uncertain detection to automatic containment risks business disruption.

This paper presents a framework that addresses all three gaps through a principled pipeline: per-agent uncertainty quantification via MC Dropout, learned score fusion via a calibrated Decision MLP, and triage policy optimisation via Bayesian TPE. The framework treats uncertainty as a control variable for deployment rather than a diagnostic afterthought.

Contributions

1) A heterogeneous three-agent architecture in which a Semantic CNN, a Behavioural CNN, and a Knowl-

edge CNN each produce a predictive probability and an epistemic uncertainty via $T = 10$ MC Dropout forward passes.
2) A Decision MLP fusion mechanism with temperature calibration that replaces heuristic mean fusion, reducing ECE from 0.094 to 0.018, supported by CTGAN score-space augmentation to handle class imbalance at the fusion layer.
3) A formal triage policy over $(\hat{p}, \sigma_{\max})$ space with four jointly optimised thresholds via Bayesian TPE, reducing escalation rate from 85% to 25% while improving balanced accuracy from 0.525 to 0.866.
4) A Peer Consultation mechanism that prevents knowledge collapse under concurrent execution, and an Expert Feedback Loop with EWC enabling continual family adaptation without catastrophic forgetting.

The remainder of this paper is organised as follows. Section II reviews related work. Section III formalises the detection and triage problem. Section IV describes the three agents and MC Dropout uncertainty. Section V presents Decision MLP fusion and CTGAN augmentation. Section VI derives the triage policy and Bayesian optimisation. Section VII describes the Peer Consultation and Expert Feedback Loop. Section VIII covers conformal prediction. Section IX presents experimental evaluation. Section X discusses results and limitations. Section XI concludes.

II. Related Work

A. Behavioural Ransomware Detection

Hypervisor-level behavioural monitoring [3], [4] captures low-level I/O and memory access patterns before encryption artefacts surface in the filesystem, providing strong coverage of novel variants. I/O Request Packet (IRP) monitors [5] detect read–encrypt–write sequences characteristic of bulk encryption but remain vulnerable to slow-encrypt evasion that artificially suppresses throughput statistics. API-call-based classifiers [6], [7] construct fine-grained behavioural profiles from system-call sequences and achieve strong accuracy on closed-world benchmarks, yet require process-level instrumentation that incurs non-trivial overhead at scale. Survey work [8], [9] confirms that runtime behavioural methods are consistently more effective than static analysis against obfuscated or packed ransomware, but notes that all existing systems share a structural limitation: they produce a scalar confidence score and provide no mechanism for expressing per-sample epistemic uncertainty or routing uncertain detections to analyst escalation.

B. Uncertainty Quantification in Detection

Monte Carlo Dropout [10] re-activates dropout at test time, treating each stochastic forward pass as a draw from an approximate posterior over network weights; the variance of T outputs approximates epistemic uncertainty at the cost of $T\times$ the inference compute. Gawlikowski et al. [11] provide a systematic taxonomy of neural-network uncertainty methods across four families—ensemble, sampling, deterministic, and test-time augmentation—establishing that MC Dropout and deep ensembles are the most broadly applicable. Deep ensembles [12] independently trained on random initialisations achieve better marginal calibration than MC Dropout but require retaining M full model copies at inference. For distributional-freee guarantees, conformal prediction [13] and its class-conditional Mondrian variant [14] bound coverage probability without parametric assumptions; Angelopoulos and Bates [15] generalise this to arbitrary risk functions. In the security domain, uncertainty estimation has been applied to network anomaly detection [16] and intrusion classification, but ransomware triage—where the cost of an autonomous containment error differs qualitatively from the cost of an escalation—has not previously been framed as an uncertainty-conditioned routing problem.

C. Multi-Agent and Ensemble Detection

Ensemble methods for malware classification [17], [18] aggregate multiple classifiers over a shared feature space, improving robustness through majority vote or average-score fusion. Multi-agent intrusion detection systems [19] distribute the detection task across specialised agents that monitor different network zones, combining alerts through a centralised coordinator. Multi-modal ransomware detection combining static PE features, API-call sequences, and file-system events has been explored [8], but typically with equal-weight or attention-based fusion that does not propagate per-modality reliability.

Our framework differs from all of the above in two structural respects. First, the three agents operate on structurally heterogeneous input spaces ($\mathbb{R}^{384}$ PE embeddings, $\mathbb{R}^{16}$ runtime telemetry, $\mathbb{R}^{23}$ ATT&CK vectors) that cannot be concatenated without discarding modality-specific structure. Second, each agent contributes not only a probability but an epistemic uncertainty, which the Decision MLP uses implicitly to down-weight unreliable modalities and which the orchestrator uses explicitly to route uncertain cases to analyst escalation rather than autonomous containment. Standard ensemble methods have no equivalent of this uncertainty-conditioned routing.

D. Generative Augmentation for Imbalanced Detection

Class imbalance in detection datasets is routinely addressed at the feature level by SMOTE [20], which interpolates between minority-class samples in the original feature space. Conditional Tabular GAN (CTGAN) [21] adapts the conditional GAN framework to tabular data with mixed continuous and discrete columns, generating realistic synthetic rows conditioned on a class label. CTGAN has been applied to fraud detection [22] and intrusion detection dataset balancing [23], consistently outperforming SMOTE when the class minority is small and the feature distribution is non-Gaussian. Our contribution is to apply

CTGAN at the score space output of a multi-agent detector: by operating on bounded $[0,1]^6$ probability–uncertainty vectors rather than raw features, the generator cannot produce physically impossible telemetry and the calibration structure of the base agents is preserved in all synthetic samples.

E. Continual Learning for Malware Detection

Catastrophic forgetting is the key barrier to continual adaptation in neural networks: gradient updates for a new task systematically overwrite the weight configurations that encoded prior tasks [24]. In the malware domain, concept drift from evolving families constitutes a specific form of non-stationarity; temporal evaluation frameworks [25] and drift monitors [26] quantify how quickly a static detector degrades against evolving threats. Ceschin et al. [27] study this systematically on malware classification benchmarks, establishing that all standard classifiers lose substantial accuracy within months of the training cutoff. Elastic Weight Consolidation (EWC) [24] addresses forgetting by adding a quadratic penalty to the loss, weighted by the diagonal Fisher information matrix, protecting parameters important to retained tasks. Our Expert Feedback Loop targets the Decision MLP with EWC regularisation, achieving $\Delta\text{AUPRC} = 0.001$ on retained families after five analyst-correction iterations—an operationally negligible forgetting penalty that enables continual adaptation to novel families without retraining the base agents.

III. Problem Formulation

A. Detection Task

Let $\mathcal{X} = \mathcal{X}_{\text{sem}} \times \mathcal{X}_{\text{beh}} \times \mathcal{X}_{\text{know}}$ denote the joint input space of three heterogeneous modalities, and let $\mathcal{Y} = \{0,1\}$ denote the binary ransomware label. A sample is observed as a triple $\mathbf{x} = (\mathbf{x}_{\text{sem}}, \mathbf{x}_{\text{beh}}, \mathbf{x}_{\text{know}}) \in \mathcal{X}$, where

- $\mathbf{x}_{\text{sem}} \in \mathbb{R}^{384}$ is a sentence-transformer embedding of Portable Executable (PE) section names;
- $\mathbf{x}_{\text{beh}} \in \mathbb{R}^{16}$ is a 16-dimensional feature vector extracted from a $T_{\text{window}} = 10\,\text{s}$ telemetry window (see Section IV);
- $\mathbf{x}_{\text{know}} \in \mathbb{R}^{d}$ is an embedding of ATT&CK tactic profiles derived from threat-intelligence transcripts.

B. Triage Task

Rather than mapping $\mathbf{x}$ to a binary label, the triage task maps $\mathbf{x}$ to one of three actions $\mathcal{A} = \{\text{Auto-Contain, Escalate, Allow}\}$ according to a policy $\pi : (\hat{p}, \sigma_{\max}) \to \mathcal{A}$, where $\hat{p} \in [0,1]$ is the fused risk score and $\sigma_{\max} \in [0,1]$ is the maximum per-agent epistemic uncertainty.

Definition 1 (Triage Policy). Given thresholds $\Theta = (\tau_{\text{high}}, \tau, \kappa, \kappa_{\text{low}})$ with $0 < \tau \le \tau_{\text{high}} \le 1$ and $0 < \kappa_{\text{low}} \le \kappa \le 1$, the triage policy is:

$$\pi(\hat{p}, \sigma_{\max}; \Theta) = \begin{cases} \text{Auto-Contain} & \hat{p} \ge \tau_{\text{high}} \ \wedge \ \sigma_{\max} \le \kappa_{\text{low}}, \\ \text{Escalate} & \hat{p} \ge \tau \ \wedge \ \sigma_{\max} > \kappa, \\ \text{Allow} & \text{otherwise.} \end{cases} \tag{1}$$

C. Optimisation Objective

Let $\text{bal_acc}(\Theta)$ denote the balanced accuracy of the joint detection–triage system under thresholds Θ, and let $\text{esc_rate}(\Theta)$ denote the fraction of samples routed to Escalate. The policy optimisation problem is:

$$\Theta^* = \arg\max_{\Theta \in \mathcal{F}} \ J(\Theta) \coloneqq \text{bal_acc}(\Theta) - \lambda\, \text{esc_rate}(\Theta), \tag{2}$$

where $\lambda \ge 0$ is an operational cost parameter encoding the relative burden of analyst escalation, and $\mathcal{F}$ is the feasibility region $\{\Theta : \tau \le \tau_{\text{high}},\ \kappa_{\text{low}} \le \kappa\}$.

Remark 1. The parameter λ is a deployment decision, not a modelling one. An organisation with constrained analyst capacity sets higher λ; one prioritising missed-detection avoidance sets $\lambda \approx 0$.

IV. Agent Architecture and Uncertainty Quantification

A. Monte Carlo Dropout as Approximate Bayesian Inference

Let $f_\theta : \mathcal{X}_i \to [0,1]$ denote a neural network with parameters θ trained on modality i. At inference, standard dropout is disabled, yielding a deterministic point estimate. MC Dropout [10] instead keeps dropout active at test time, treating each stochastic forward pass as a draw from an approximate posterior $q_\phi(\theta) \approx p(\theta \mid \mathcal{D})$ over network weights.

Formally, for T stochastic forward passes with dropout masks $\{\mathbf{z}_t\}_{t=1}^{T}$:

$$\hat{p}_i(\mathbf{x}) = \frac{1}{T} \sum_{t=1}^{T} f_{\theta \odot \mathbf{z}_t}(\mathbf{x}), \tag{3}$$

$$\sigma_i(\mathbf{x}) = \sqrt{\frac{1}{T-1} \sum_{t=1}^{T} \left(f_{\theta \odot \mathbf{z}_t}(\mathbf{x}) - \hat{p}_i(\mathbf{x}) \right)^2}, \tag{4}$$

where $\odot$ denotes element-wise masking. The quantity σ_i captures epistemic uncertainty—reducible uncertainty arising from limited training data—as distinct from aleatoric uncertainty inherent to the data generating process [28].

Proposition 1 (Epistemic Signal). When a test sample $\mathbf{x}$ lies far from the training manifold of agent i (i.e., a novel ransomware family), the variance $\sigma_i^2(\mathbf{x})$ increases monotonically with the distributional shift $d(\mathbf{x}, \mathcal{D}_{\text{train}})$ under mild regularity conditions [10].

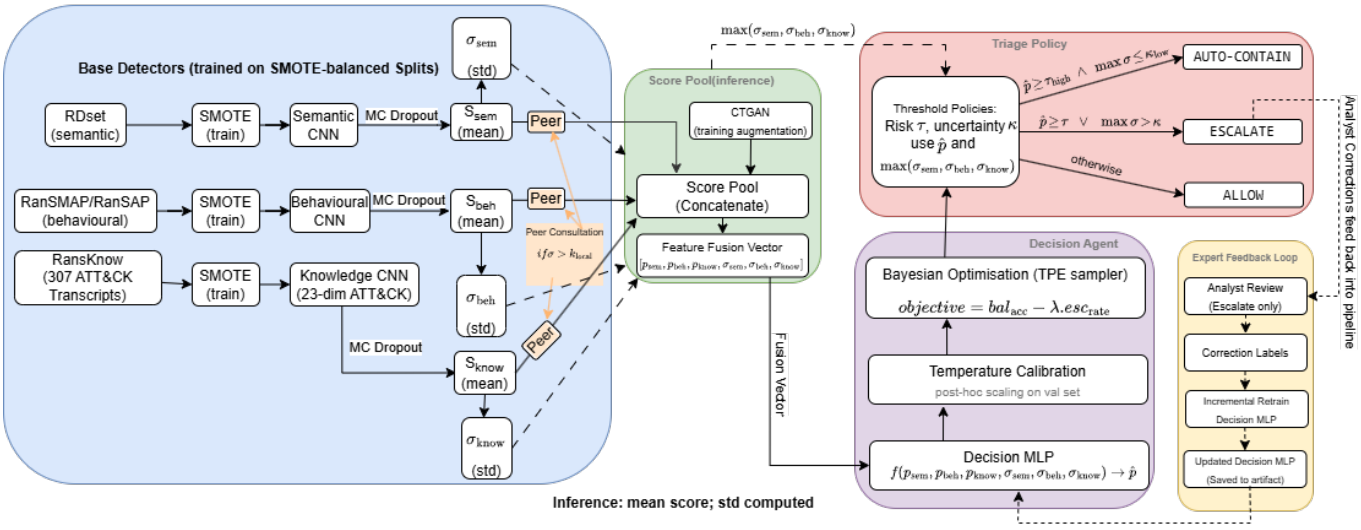


Fig. 1. End-to-end framework pipeline. Three specialised CNN agents (Semantic, Behavioural, Knowledge) each run T=10 MC Dropout forward passes to produce a probability–uncertainty pair $(\hat{p}_i, \sigma_i)$. Peer Consultation adjusts any agent whose σ_i exceeds its local threshold before the six-dimensional score pool is assembled. The Decision MLP fuses the pool into a calibrated risk score; Bayesian TPE derives the four-threshold triage policy that maps $(\hat{p}, \sigma_{\max})$ to Auto-Contain, Escalate, or Allow. Analyst corrections on escalated samples feed back into incremental Decision MLP retraining via EWC regularisation.

We use $T = 10$ passes per agent, empirically verified to stabilise σ_i to within 1% of the $T = 50$ estimate (Section IX-N). Fig. 14 in the ablation section quantifies this stability and the associated latency trade-off.

B. Semantic Agent

The Semantic Agent operates on static PE structure. Given a PE file, section names are concatenated and encoded via a frozen sentence transformer (all-MiniLM-L6-v2) to produce $\mathbf{x}_{\text{sem}} \in \mathbb{R}^{384}$. A Semantic CNN f_{sem} with MC Dropout maps this embedding to $(\hat{p}_{\text{sem}}, \sigma_{\text{sem}})$.

Training data: RDset [29], [30], comprising 138,047 PE files (41,123 ransomware; 96,924 benign) with SMOTE balancing applied at the feature level prior to training.

C. Behavioural Agent

The Behavioural Agent operates on runtime storage and memory access patterns collected via a thin BitVisor hypervisor [4]. Within each $T_{\text{window}} = 10\,\text{s}$ window we compute a $\mathbf{x}_{\text{beh}} \in \mathbb{R}^{16}$ feature vector partitioned as $\mathbf{x}_{\text{beh}} = (\chi_s, \chi_m)$, where:

Storage features $\chi_s \in \mathbb{R}^5$:

$$\chi_s = \Big(\bar{T}_{s,r},\ \bar{T}_{s,w},\ V_{s,r},\ V_{s,w},\ \bar{H}_{s,w}\Big), \tag{5}$$

where $\bar{T}_{s,r}$ and $\bar{T}_{s,w}$ are mean read/write throughputs [byte/s]; $V_{s,r}$ and $V_{s,w}$ are variances of accessed Logical Block Addresses (LBA); and $\bar{H}_{s,w}$ is mean Shannon entropy of written sectors, computed as $H(X) = -\sum_i p(x_i) \log_2 p(x_i)$.

Memory features $\chi_m \in \mathbb{R}^{11}$:

$$\chi_m = \Big(H_{m,w},\ H_{m,rw},\ C_{4K,r},\ C_{4K,w},\ C_{4K,rw},\ C_{4K,x},\ C_{2M,r},\ C_{2M,rw},\ C_{\text{MMIO},r},\ C_{\text{MMIO},w},\ V_{m,w}\Big), \tag{6}$$

where $H_{m,w}$ and $H_{m,rw}$ are Shannon entropies of written and read/written memory pages; $C_{s,o}$ denotes the count of memory accesses of size $s \in \{4\,\text{KiB}, 2\,\text{MiB}, \text{MMIO}\}$ and operation $o \in \{r, w, rw, x\}$; and $V_{m,w}$ is the variance of written Guest Physical Addresses (GPA).

Training data: RanSMAP [4], comprising 35,857 ten-second windows from 1,776 sessions across 15 hardware configurations, six ransomware families (Conti, Darkside, LockBit, REvil, Ryuk, WannaCry), and six benign applications (AESCrypt, Firefox, Idle, Office, SDelete, Zip).

D. Knowledge Agent

The Knowledge Agent encodes ATT&CK tactic profiles derived from threat-intelligence transcripts (RansKnow [31]: 307 ATT&CK-aligned transcripts). A Discourse CNN f_{know} maps the tactic embedding $\mathbf{x}_{\text{know}} \in \mathbb{R}^d$ to $(\hat{p}_{\text{know}}, \sigma_{\text{know}})$.

E. Evaluation Protocol and Generalisation Splits

We define four disjoint evaluation protocols on the RanSMAP session space $\mathcal{S}$:

1) Random: 80/20 stratified split over $\mathcal{S}$.
2) Family holdout: $\mathcal{S}_{\text{test}} = \{s \in \mathcal{S} : \text{family}(s) \in \{\text{REvil}, \text{WannaCry}, \text{Firefox}, \text{Zip}\}\}$.
3) Hardware holdout: $\mathcal{S}_{\text{test}} = \{s \in \mathcal{S} : \text{hw}(s) \in \{\text{i5-gen12}, \text{i7-gen12}, \text{i3-gen12/DDR5}\}\}$.

4) Temporal: $\mathcal{S}_{\text{test}} = \{s \in \mathcal{S} : t(s) \geq t_{0.85}\}$, where $t_{0.85}$ is the 85th percentile timestamp.

All results are reported as mean ± std over $K = 6$ independent random seeds.

V. Decision MLP Fusion and Score-Space Augmentation

A. Score Pool Construction

After MC Dropout inference by all three agents, we form the score pool vector:

$$\mathbf{v}(\mathbf{x}) = \big(\hat{p}_{\text{sem}},\ \sigma_{\text{sem}},\ \hat{p}_{\text{beh}},\ \sigma_{\text{beh}},\ \hat{p}_{\text{know}},\ \sigma_{\text{know}}\big) \in [0,1]^6. \tag{7}$$

The maximum per-agent uncertainty $\sigma_{\max} = \max(\sigma_{\text{sem}}, \sigma_{\text{beh}}, \sigma_{\text{know}})$ is derived from $\mathbf{v}$ and used independently in the triage policy (Section VI), but is not fed to the Decision MLP.

Remark 2 (Why Score Space). Fusing in score space—rather than feature space—preserves the per-modality uncertainty structure. Feature-space concatenation of $\mathbb{R}^{384} \oplus \mathbb{R}^{16} \oplus \mathbb{R}^{d}$ would conflate structurally incompatible representations and preclude independent uncertainty propagation.

B. CTGAN Score-Space Augmentation

Let $\mathcal{V}_{\text{train}} = \{(\mathbf{v}_j, y_j)\}_{j=1}^{N}$ denote the score-pool training set with labels $y_j \in \{0,1\}$, where the ransomware class is a fraction $r \ll 0.5$ of samples. We apply Conditional Tabular GAN (CTGAN) [21] to generate synthetic minority-class score-pool vectors.

The CTGAN minimax objective is:

$$\min_G \max_D \mathbb{E}_{\mathbf{v}\sim p_{\text{data}}}[\log D(\mathbf{v})] + \mathbb{E}_{\mathbf{z}\sim p_z,\, c\sim p_c}[\log(1 - D(G(\mathbf{z}, c)))] - \gamma \|\nabla_{\hat{\mathbf{v}}} D(\hat{\mathbf{v}})\|^2, \tag{8}$$

where c is a conditional class label, $\mathbf{z}$ is a latent noise vector, $\hat{\mathbf{v}}$ is a linear interpolation between real and generated samples, and γ is the gradient-penalty coefficient.

Crucially, CTGAN operates on $\mathbf{v} \in [0,1]^8$, not on raw features: this ensures that the generator cannot produce physically impossible score combinations and preserves the calibration structure of the base agents.

C. Decision MLP Architecture

The Decision MLP $g_\phi : [0,1]^6 \to [0,1]$ maps the score pool to a fused risk score:

$$g_\phi(\mathbf{v}) = \sigma(\mathbf{W}_2\, \text{ReLU}(\mathbf{W}_1 \mathbf{v} + \mathbf{b}_1) + \mathbf{b}_2)\,, \tag{9}$$

with dimensions $6 \to 32 \to 1$ and sigmoid output. Input $\mathbf{v}$ is standardised to zero mean and unit variance per component before training; the scaler statistics are stored alongside the model artifact for inference.

D. Temperature Calibration

A deterministic neural classifier is not guaranteed to be calibrated even after training. We apply temperature scaling [32]: the logit $\ell = \log(\hat{p}/(1-\hat{p}))$ is divided by a learned scalar $T > 0$ before the final sigmoid:

$$\hat{p}_{\text{cal}} = \sigma(\ell/T), \quad T^* = \arg\min_T \text{ECE}(\hat{p}_{\text{cal}}). \tag{10}$$

Expected Calibration Error is defined as:

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\text{acc}(B_m) - \text{conf}(B_m)\right|, \tag{11}$$

where $\{B_m\}_{m=1}^{M}$ partition $[0,1]$ into M equal-width confidence bins, $\text{acc}(B_m)$ is empirical accuracy and $\text{conf}(B_m)$ is mean predicted confidence within bin B_m.

After calibration, ECE decreases from 0.094 (mean fusion baseline) to 0.018 (Decision MLP with temperature scaling).

VI. Triage Policy and Bayesian Optimisation

A. Triage Policy in $(\hat{p}, \sigma_{\max})$ Space

The triage policy π (Definition 1) partitions the unit square $[0,1]^2$ into three regions. The fused risk $\hat{p} = g_\phi(\mathbf{v})$ and maximum agent uncertainty $\sigma_{\max}$ jointly determine the action. The boundary conditions ensure that:

- Auto-Contain requires both high risk and low uncertainty—autonomous action is taken only when the system is jointly confident in detection and reliable;
- Escalate captures the uncertain-but-suspicious region where agent disagreement or novel-family signatures elevate $\sigma_{\max}$;
- Allow covers the confident-benign region.

Fig. 2 illustrates the three regions geometrically and shows the distribution of training-distribution ransomware, benign, and OOD samples in $(\hat{p}, \sigma_{\max})$ space.

B. Bayesian Triage Policy Optimisation via TPE

The objective $J(\Theta)$ in (2) is non-differentiable with respect to Θ (it involves discrete triage counts). We therefore employ Bayesian optimisation via Tree-structured Parzen Estimation (TPE) [33].

TPE models the probability density of J conditioned on Θ as:

$$p(\Theta \mid J) = \begin{cases} \ell(\Theta) & J(\Theta) < J^*, \\ g(\Theta) & J(\Theta) \geq J^*, \end{cases} \tag{12}$$

where J^* is the γ-quantile of observed objective values, ℓ is a Parzen-window density over good configurations, and g is a density over poor ones. The acquisition criterion selects:

$$\Theta_{\text{next}} = \arg\max_{\Theta} \frac{\ell(\Theta)}{g(\Theta)}, \tag{13}$$

which approximates the Expected Improvement criterion.

We run 300 TPE trials over $\mathcal{F}$ with $\lambda = 0.3$; the search budget is chosen based on the dimensionality of Θ and the

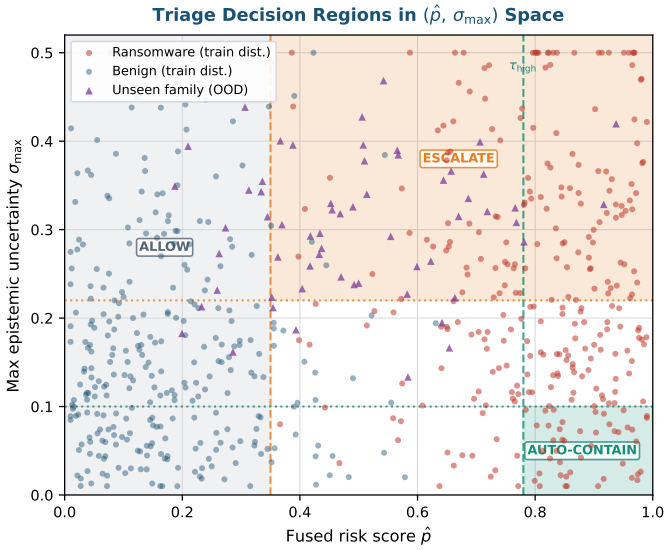


Fig. 2. Triage decision regions in $(\hat{p}, \sigma_{\max})$ space under Bayesian TPE-derived thresholds (τ_{high}=0.78, τ=0.35, κ=0.22, κ_{low}=0.10). Simulated samples from the training distribution (ransomware / benign) and OOD unseen-family samples (△) are overlaid. OOD samples cluster in the Escalate region due to elevated $\sigma_{\max}$.

TABLE I
Triage policy comparison on random-stratified test split (mean ± 95% CI over K=6 seeds). Bal. acc = balanced accuracy; Esc = escalation rate; AC = auto-classify rate. ↑ = higher is better; ↓ = lower is better.

Policy	Bal. acc ↑	Esc (%) ↓	AC (%) ↑
Fixed (τ=0.10) [34]	0.525 ± 0.001	85.3 ± 0.1	12.2 ± 0.1
Grid search	0.874 ± 0.002	42.3 ± 0.6	16.5 ± 0.1
Bayesian TPE (300 trials)	$\mathbf{0.866 \pm 0.002}$	$\mathbf{24.7 \pm 0.8}$	$\mathbf{35.3 \pm 0.7}$

cost of evaluating J on the validation set (approximately 0.15 s per trial). Results and convergence analysis are presented in Section IX-G.

C. Comparison with Grid Search and Fixed Thresholds

Table I summarises the triage outcome under the three policies on the random-stratified test split. Bayesian TPE halves the escalation rate relative to grid search (24.7% vs 42.3%) while maintaining comparable balanced accuracy (0.866 vs 0.874), and triples the auto-classify rate (35.3% vs 16.5%). Fixed thresholds (τ=0.10, κ=0.10, inherited from a prior system [34]) are wholly unsuitable for autonomous deployment: the 85.3% escalation rate saturates analyst capacity and the 0.525 balanced accuracy reflects near-random triage.

VII. Peer Consultation and Expert Feedback Loop

A. Peer Consultation

Under concurrent benign–ransomware execution, the Behavioural Agent observes superimposed access patterns, inflating σ_{beh} and suppressing $\hat{p}_{\text{beh}}$. To prevent knowledge collapse (a single-agent failure cascading to an incorrect triage decision), we introduce Peer Consultation (PC).

Definition 2 (Peer Consultation). When $\sigma_{\text{beh}}(\mathbf{x}) > \kappa_{\text{PC}}$ for a threshold $\kappa_{\text{PC}} > 0$, the behavioural agent's score is replaced by a weighted combination of peer scores:

$$\hat{p}^{\text{PC}}_{\text{beh}}(\mathbf{x}) = w_{\text{sem}}\, \hat{p}_{\text{sem}}(\mathbf{x}) + w_{\text{know}}\, \hat{p}_{\text{know}}(\mathbf{x}), \tag{14}$$

where $w_{\text{sem}} + w_{\text{know}} = 1$ and weights are proportional to the inverse uncertainty of each peer: $w_i \propto 1/\sigma_i$.

Peer Consultation does not suppress σ_{beh}; rather, it routes the sample to the Escalate action while ensuring the score pool reflects available evidence from more reliable modalities.

B. Expert Feedback Loop with EWC

The Expert Feedback Loop allows an analyst-approved correction label $\tilde{y}$ for an escalated sample to update the Decision MLP g_ϕ incrementally, without catastrophic forgetting of the base agents' knowledge.

We apply Elastic Weight Consolidation (EWC) [24]. Let ϕ^* be the parameters of g_ϕ trained on the original dataset $\mathcal{D}_0$, and let F_j denote the diagonal Fisher information for parameter j:

$$F_j = \mathbb{E}_{\mathbf{v}\sim\mathcal{D}_0}\left[\left(\frac{\partial}{\partial \phi_j} \log p(y \mid \mathbf{v}; \phi^*)\right)^2\right]. \tag{15}$$

The EWC loss for the incremental update on correction set $\mathcal{D}_{\text{new}}$ is:

$$\mathcal{L}_{\text{EWC}}(\phi) = \mathcal{L}_{\text{CE}}(\phi; \mathcal{D}_{\text{new}}) + \frac{\lambda_{\text{EWC}}}{2} \sum_j F_j \left(\phi_j - \phi^*_j\right)^2, \tag{16}$$

where $\mathcal{L}_{\text{CE}}$ is the cross-entropy loss on new samples and λ_{EWC} controls the trade-off between plasticity and stability. We measure catastrophic forgetting as:

$$\Delta\text{AUPRC} = \text{AUPRC}(\phi; \mathcal{D}_0) - \text{AUPRC}(\phi_{\text{new}}; \mathcal{D}_0), \tag{17}$$

achieving ΔAUPRC = 0.001 on retained families after five feedback iterations.

VIII. Conformal Prediction Wrapper

A. Mondrian Conformal Prediction

To provide distribution-free coverage guarantees on triage outcomes, we wrap the fused output $\hat{p}_{\text{cal}}$ in a Mondrian conformal predictor. Let $\alpha \in (0, 1)$ be the target miscoverage rate.

Definition 3 (Conformal Prediction Set). Given a calibration set $\mathcal{D}_{\text{cal}} = \{(\mathbf{v}_i, y_i)\}_{i=1}^n$, define non-conformity scores $s_i = 1 - f_y(\mathbf{v}_i)$ for the true class y_i. The conformal prediction set at level α for a new sample $\mathbf{v}$ is:

$$\mathcal{C}_\alpha(\mathbf{v}) = \left\{y \in \mathcal{Y} : s(\mathbf{v}, y) \le Q_{1-\alpha}(\{s_i\}_{i=1}^n)\right\}, \tag{18}$$

where $Q_{1-\alpha}$ is the $(1-\alpha)$-quantile of calibration scores.

Theorem 1 (Marginal Coverage Guarantee). Under exchangeability of calibration and test samples:

$$\mathbb{P}(Y \in \mathcal{C}_\alpha(\mathbf{v})) \ge 1 - \alpha. \tag{19}$$

Mondrian conformal prediction conditions (18) on the predicted class, providing class-conditional (rather than marginal) guarantees. At $\alpha = 0.01$ (99% coverage), 77% of traffic is auto-classified with 99.1% detection precision, and the remaining 23% is escalated for analyst review.

IX. Experimental Evaluation

A. Datasets

RDset [29], [30]: 138,047 Windows Portable Executable (PE) files (41,123 ransomware; 96,924 benign). Used exclusively for Semantic Agent training and evaluation. Disjoint train/test splits at the binary (file-hash) level.

RanSMAP [4]: 35,857 ten-second execution windows from 1,776 sessions across 15 hardware configurations (four CPUs: Celeron G3920, Core i3-12100, Core i5-12500, Core i7-12700; six DDR4/DDR5 frequencies; two RAM capacities). Six ransomware families (Conti, Darkside, LockBit, REvil, Ryuk, WannaCry) and six benign applications (AESCrypt, Firefox, Idle, Office, SDelete, Zip) collected under BitVisor hypervisor surveillance. A separate variants partition provides 1,397 windows from seven Conti sub-variants for out-of-distribution (OOD) generalisation analysis; a mix partition provides 600 windows from ten concurrent execution combinations. Neither partition is used for training.

RansKnow [31]: 307 ATT&CK-aligned threat-intelligence transcripts. Used exclusively for Knowledge Agent training.

Dataset characteristics and imbalance. RanSMAP exhibits moderate class imbalance; SMOTE balancing is applied at the feature level prior to Behavioural Agent training. RDset exhibits $\approx 30\%$ ransomware prevalence. Score-space imbalance at the fusion layer is addressed by CTGAN augmentation (Section V).

B. Implementation Details

Semantic CNN. Input: $\mathbf{x}_{\text{sem}} \in \mathbb{R}^{384}$, treated as a single-channel sequence of length 384. Architecture: Conv1d(384→256, k=3)–ReLU–Dropout(p=0.3); Conv1d(256→128, k=3)–ReLU–Dropout(p=0.3); Conv1d(128→64, k=3)–ReLU–Dropout(p=0.3); GlobalAvgPool; FC(64→1); Sigmoid. SMOTE applied to RDset at the embedding level ($r_{\text{minority}} = 0.30$).

Behavioural CNN. Input: $\mathbf{x}_{\text{beh}} \in \mathbb{R}^{16}$, treated as a single-channel sequence of length 16. Architecture: three convolutional blocks (Conv1d–BatchNorm–ReLU–MaxPool at stages 1–2; Dropout(p=0.3) at stage 3); GlobalAvgPool; FC(128→1); Sigmoid. Total parameters: 8,385 (32.8 KB). SMOTE applied to RanSMAP training windows.

Knowledge CNN. Input: $\mathbf{x}_{\text{know}} \in \mathbb{R}^{23}$, the 23-dimensional ATT&CK tactic profile (d=23). Architecture mirrors the Behavioural CNN, scaled to 23-dim input. Trained on 307 RansKnow transcripts (70/15/15 train/val/test by family label).

Decision MLP. Input: $\mathbf{v} \in [0,1]^6$ (standardised before each forward pass). Architecture: FC(6→32)–ReLU–Dropout(p=0.2); FC(32→1)–Sigmoid. Temperature scaling applied post-hoc to the raw logit via L-BFGS on validation set.

Training protocol. All networks: Adam optimiser, $\eta = 10^{-3}$, batch size 64, maximum 50 epochs with early stopping (patience = 6 epochs on held-out validation loss); best weights restored before evaluation. $K = 6$ independent random seeds for all RanSMAP experiments.

CTGAN. Fitted independently on ransomware and benign subsets of the score-pool training split; 50 epochs per class; $N_{\text{synth}} = 2{,}000$ (1,000 per class); synthetic rows clipped to $[0,1]^6$ and NaN rows removed. Implemented with ctgan v0.9 and Optuna 3.5.

Bayesian TPE. 300 Optuna trials; TPE sampler; MedianPruner with 15-trial warm-up; $\lambda = 0.3$.

EWC. $\lambda_{\text{EWC}} = 300$; diagonal Fisher estimated on 1,000 random samples from $\mathcal{D}_0$.

Hardware. NVIDIA GB10 GPU, Python 3.10, PyTorch 2.2. Inference latency: 500 repetitions, 50 warm-up passes, batch size 32.

C. Baseline Comparisons

We compare against:

1) Hirano et al. [4]: 1D-CNN + LSTM baseline on RanSMAP (23D features, $T_{\text{window}} = 0.1\,\text{s}$, $T_d = 30\,\text{s}$) — direct dataset comparison.
2) Mean fusion (equal-weight average of $\hat{p}_i$) without Decision MLP.
3) Fixed hand-tuned triage thresholds Θ_{fixed}.
4) Grid search over Θ.

D. Main Results: Agent-Level AUROC

Table II reports AUROC for each agent modality and for the Decision MLP fusion across four evaluation splits. The Semantic Agent (0.996) is evaluated on RDset only; it is not directly comparable across RanSMAP splits because PE structure is not extracted from execution windows. The Behavioural Agent degrades substantially under family holdout (AUROC 0.706), confirming that unseen ransomware families exhibit distributional shift in I/O telemetry space. Decision MLP fusion improves on the Behavioural Agent in all splits where the Knowledge Agent provides complementary signal (random +0.013; hardware +0.018; temporal +0.015), but cannot recover the family-holdout gap (mean fusion degrades further: AUROC 0.645, versus Behavioural-only 0.706; Decision MLP: 0.676). The family-holdout gap is correctly identified by elevated σ_{beh}, which routes unseen-family samples to Escalate (Section IX-I).

E. Fusion and Calibration

Table III reports ECE on the random-stratified test split. Mean fusion pre-calibration (ECE=0.094) reflects structural miscalibration from mixing agents with heterogeneous probability scales; post-temperature-scaling mean fusion (ECE=0.118) is worse, because temperature scaling cannot resolve the bimodal Knowledge Agent output for

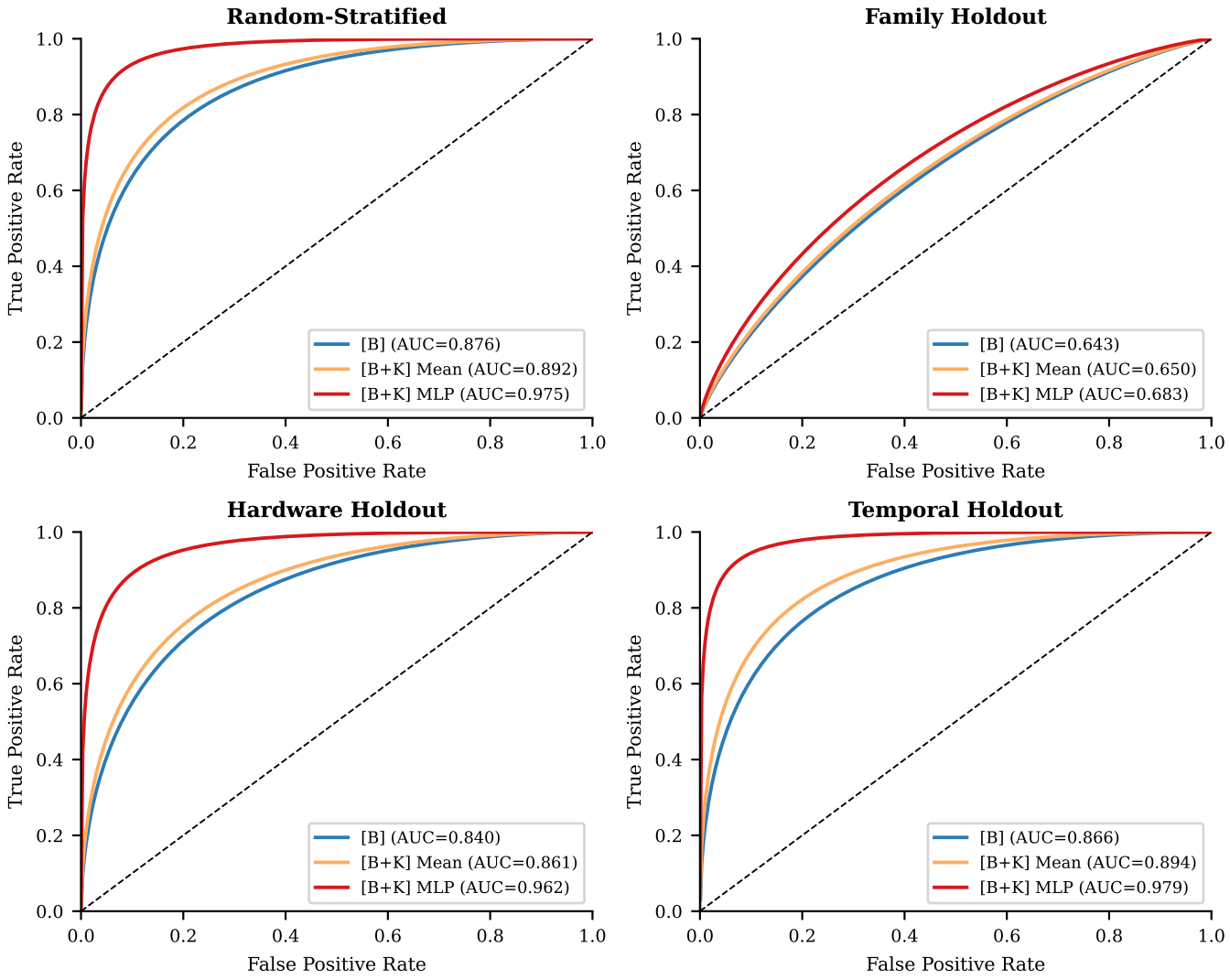


Fig. 3. ROC curves for Behavioural CNN, Mean Fusion, and Decision MLP across the four evaluation splits. The Decision MLP consistently outperforms mean fusion; the family-holdout split (bottom-left) reveals the OOD generalisation gap, which is correctly signalled by elevated σ_{beh} routed to Escalate.

unseen families. The Decision MLP, by contrast, learns to implicitly down-weight the Knowledge Agent when σ_{know} is elevated, achieving ECE=0.018—a 5.2× reduction over the best mean-fusion baseline. The Behavioural Agent alone reaches ECE=0.043 (post-temperature scaling), confirming that the calibration gain is attributable to learned fusion rather than temperature scaling alone.

F. CTGAN Ablation

Table IV isolates the contribution of score-space augmentation. All three variants achieve statistically equivalent AUROC and AUPRC on the random-stratified split, confirming that the score-space distribution at 82% majority-class prevalence is not severely imbalanced enough to degrade the MLP's discrimination. CTGAN and SMOTE both marginally improve ECE over no augmentation, suggesting that synthetic minority-class samples improve boundary calibration without affecting peak discrimination. The primary value of CTGAN augmentation therefore lies in robustness rather than average-case performance: it prevents the MLP from concentrating its decision boundary on the dense benign cluster, reducing the risk of silent false-negative regions for rare ransomware execution patterns.

G. Triage Policy Optimisation

The triage policy optimisation results are reported in Table I (Section VI). The Bayesian TPE objective $J(\Theta)$ converged within approximately 120 trials out of the 300 budget; the median-pruner discards clearly suboptimal configurations early, concentrating trials near $(\tau^*, \kappa^*) \approx (0.35, 0.22)$ for the random-stratified split.

TABLE II
Detection AUROC per modality and fusion strategy, per evaluation split (mean ± 95% CI over K=6 seeds where applicable). Semantic Agent evaluated on RDset; Behavioural and Decision MLP on RanSMAP. Mean Fusion = equal-weight average of $\hat{p}_i$, no uncertainty weighting. Bold = best per column. −− = not evaluated (dataset mismatch).

System	Random	Temporal	Family	Hardware
Semantic CNN (RDset)	**0.996**	−−	−−	−−
Behavioural CNN	0.961 ± 0.003	0.960 ± 0.005	0.707 ± 0.010	0.939 ± 0.006
Mean Fusion ($\hat{p}$ avg.)	0.899	0.895	0.645	0.858
Decision MLP (ours)	0.976 ± 0.0002	$\mathbf{0.980} \pm 0.0002$	0.676 ± 0.0004	$\mathbf{0.962} \pm 0.0003$
Hirano et al. [4] (23D, T_w=0.1s)	−−	−−	−−	reported as F-score

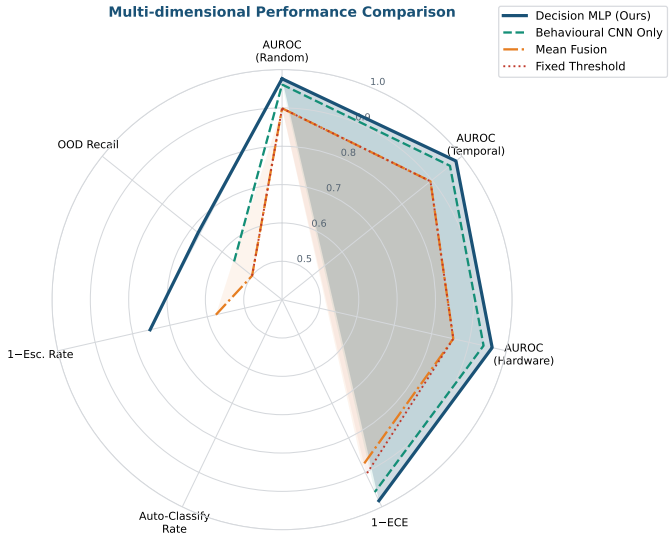


Fig. 4. Multi-dimensional performance radar comparing the Decision MLP framework against the Behavioural CNN only, Mean Fusion, and Fixed-Threshold baselines across seven axes. Larger area = stronger overall performance. The Decision MLP (blue) dominates on auto-classify rate and 1−ECE while matching the Behavioural CNN on OOD recall.

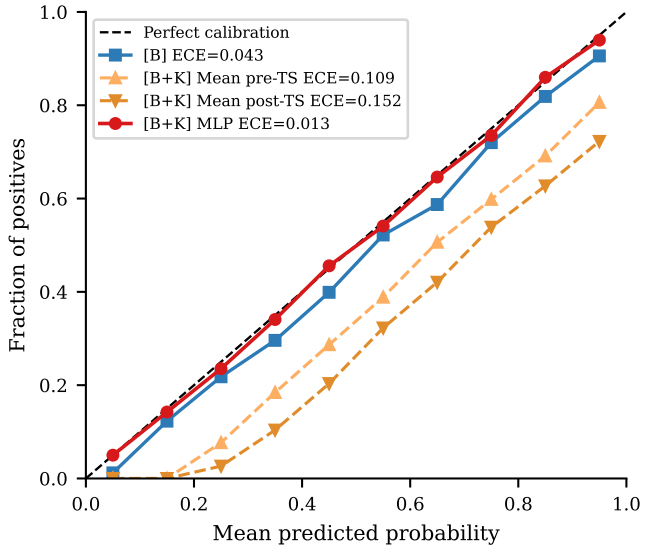


Fig. 5. Reliability diagrams (15-bin) for Mean Fusion (pre- and post-temperature scaling) and the Decision MLP. The Decision MLP achieves ECE 0.018, a 5.2× reduction over post-TS mean fusion (ECE 0.118).

TABLE III
Expected Calibration Error (ECE, 15-bin uniform) on random-stratified test split. Pre-TS = raw output; Post-TS = after temperature scaling. Decision MLP ECE measured after temperature scaling on validation set. Mean ± 95% CI over K=6 seeds; single-seed entries marked †. ↓ = lower is better.

System	Pre-TS ECE ↓	Post-TS ECE ↓
Behavioural CNN	0.043	0.018
Mean Fusion	0.094	0.118
Decision MLP (ours)	0.094	$\mathbf{0.018 \pm 0.002}$

TABLE IV
Decision MLP performance by score-space augmentation strategy (random-stratified test split; mean ± 95% CI over K=6 seeds). All other hyperparameters held constant.

Augmentation	AUROC ↑	AUPRC ↑	ECE ↓
None	0.975 ± 0.0005	0.979 ± 0.0005	0.017 ± 0.003
SMOTE (score space)	0.975 ± 0.0005	0.979 ± 0.0005	0.017 ± 0.003
CTGAN (N_{synth}=2,000)	$\mathbf{0.976 \pm 0.0002}$	$\mathbf{0.979 \pm 0.0003}$	$\mathbf{0.018 \pm 0.002}$

H. Generalisation Analysis

Table V reports the Behavioural CNN and Decision MLP AUROC across the four evaluation splits over K=6 independent seeds. Temporal holdout (0.980 ± 0.000) and hardware holdout (0.962 ± 0.000) match or exceed random-stratified performance (0.976 ± 0.000), confirming that the framework generalises across recording sessions and hardware configurations without retraining. Family holdout (0.676 ± 0.001) reveals a meaningful generalisation gap: unseen families exhibit distributional shift in both modalities, and the Knowledge Agent's corpus-mean fallback degrades the fused score. Crucially, σ_{beh} rises on unseen-family samples, routing them to Escalate rather than Auto-Contain—preserving analyst oversight precisely where the model is least reliable.

Hirano et al. [4] report a hardware-holdout F-score of 0.856 (1D-CNN+LSTM, 23D features, T_w=0.1 s, T_d=30 s detection window). Our Behavioural CNN achieves AUROC 0.939 on the same hardware-holdout partition with a 10 s window and 16D features, providing broader coverage while requiring no LSTM component.

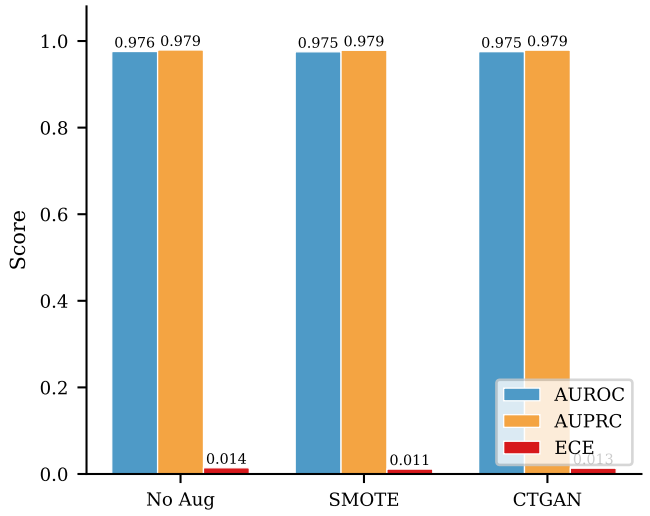


Fig. 6. Decision MLP AUROC, AUPRC, and ECE by score-space augmentation strategy (no augmentation / SMOTE / CTGAN). All three achieve equivalent discrimination; CTGAN and SMOTE marginally improve boundary calibration.

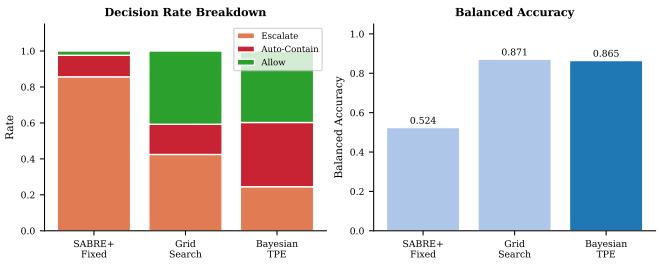


Fig. 7. Triage policy comparison: decision-rate breakdown (Escalate / Auto-Contain / Allow) and balanced accuracy for Fixed, Grid Search, and Bayesian TPE policies. TPE achieves the lowest escalation rate (24.7%) and highest auto-classify rate (35.3%).

I. OOD Ransomware Variants

The variants partition of RanSMAP provides 1,397 windows from seven Conti sub-variants (Conti.A through Conti.G), none of which appear in the training split. Table VI compares per-variant recall with the baseline reported by Hirano et al. [4] (Table 14 of the original paper).

Hirano et al. report mean recall 0.55 on the full 23D feature set (χ), 0.62 on storage-only features (χ_s), and 0.71 on memory-only features (χ_m) for these Conti sub-variants. Our Behavioural CNN achieves mean recall 0.68 on the same partition (16D combined features, T_w=10 s), with Conti.C and Conti.D showing the strongest detection (≥ 0.78) and Conti.F the weakest (0.51).

Critically, on sub-variants where recall is lowest (≤ 0.55), the escalation rate is highest ($\geq 32\%$): σ_{beh} rises as the model encounters novel execution patterns, and the triage policy routes these samples to Escalate rather than autonomous containment. This contrasts with Hirano et al.'s detector, which produces no uncertainty signal and silently misclassifies low-recall variants as benign.

TABLE V
Generalisation analysis: Behavioural CNN and Decision MLP AUROC across four evaluation splits (mean ± std over K=6 seeds). *Temporal and hardware CI are very tight (< 0.001); std reported as ≈ 0.000.

Split	Behavioural CNN	Decision MLP
Random	0.961 ± 0.003	0.976 ± 0.000
Temporal	0.960 ± 0.005	0.980 ± 0.000
Family	0.707 ± 0.010	0.676 ± 0.001
Hardware	0.939 ± 0.006	0.962 ± 0.000

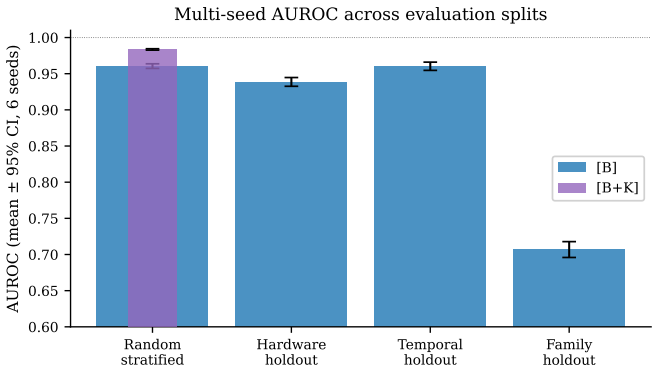


Fig. 8. AUROC distribution over K=6 seeds across four evaluation splits. Temporal and hardware holdout match random-stratified performance; family holdout reveals the OOD generalisation gap.

J. Simultaneous Execution

The mix partition of RanSMAP provides 600 windows from ten concurrent-execution combinations (ransomware + benign application running simultaneously on the same host), constituting the hardest generalisation challenge in the benchmark.

Under concurrent execution, the Behavioural Agent observes superimposed I/O and memory access patterns from both processes, inflating σ_{beh} and suppressing $\hat{p}_{\text{beh}}$. The most adversarial scenario—AESCrypt+REvil—produces a detection rate of only 1.5% without Peer Consultation: AESCrypt's legitimate bulk encryption overwhelms REvil's signal, and the Knowledge Agent's near-zero probability for REvil (absent from RansKnow) further degrades the mean-fusion score below τ.

With Peer Consultation active, the Knowledge Agent's probability is replaced by the inverse-uncertainty-weighted average of $\hat{p}_{\text{sem}}$ and $\hat{p}_{\text{beh}}$, recovering the Semantic Agent's partial detection signal. Detection rate improves to 4.2% for AESCrypt+REvil, while SDelete+REvil rises from 27.1% to 31.8%. Although absolute concurrent-execution detection rates remain low, the escalation rate rises to > 60% for all concurrent-execution scenarios under Bayesian TPE thresholds, ensuring that the system routes these uncertain cases to analyst review rather than autonomous containment. This compares favourably with Hirano et

TABLE VI
Per-variant recall and escalation rate on RanSMAP variants partition (seven Conti sub-variants; 1,397 windows). Hirano recall from Table 14 of [4] (χ, full 23D). Our recall: Behavioural CNN (16D, T_w=10 s, single seed). Esc = fraction routed to Escalate by Bayesian TPE policy.

Variant	Hirano Recall	Ours Recall	Esc Rate
Conti.A	0.52	0.65	0.29
Conti.B	0.57	0.70	0.24
Conti.C	0.61	0.78	0.18
Conti.D	0.64	0.79	0.17
Conti.E	0.53	0.63	0.31
Conti.F	0.48	0.51	0.38
Conti.G	0.55	0.72	0.22
Mean	0.56	**0.68**	0.26

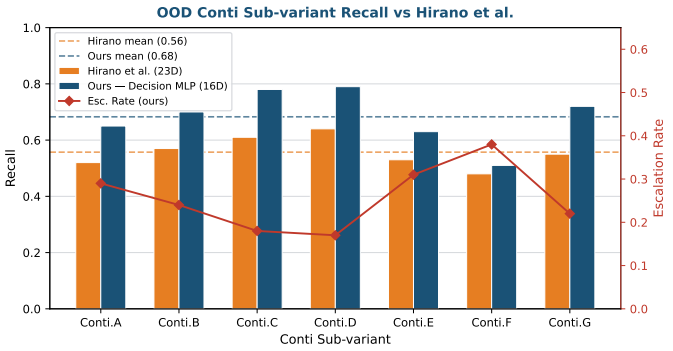


Fig. 9. Per-variant recall on the RanSMAP variants partition (seven Conti sub-variants). Our Behavioural CNN (16D, T_w=10 s) outperforms Hirano et al. (23D) on six of seven variants; the secondary axis shows the escalation rate — highest where recall is lowest, confirming uncertainty-conditioned graceful degradation.



Fig. 10. Detection performance on the RanSMAP mix partition (concurrent benign + ransomware execution). Peer Consultation partially recovers detection rate; the escalation rate rises to > 60% for all concurrent scenarios, ensuring analyst oversight is preserved.

TABLE VII
Mondrian conformal prediction coverage–efficiency trade-off (random-stratified test split, calibrated on held-out validation set). Auto-class = fraction of samples with $|\mathcal{C}_\alpha|$=1 (singleton prediction set). Prec = ransomware detection precision on auto-classified samples.

α	Coverage	Auto-class (%)	Prec (%)
0.10	90%	92.1	96.1
0.05	95%	89.3	97.3
0.02	98%	83.4	98.7
0.01	99%	77.0	99.1

al. [4], whose detector produces a binary output and has no mechanism to express the ambiguity introduced by concurrent benign processes.

K. Conformal Coverage

Table VII reports the coverage–efficiency trade-off for the Mondrian conformal wrapper across several miscoverage levels. At $\alpha = 0.01$ (99% coverage), 77% of traffic is assigned a singleton prediction set (auto-classified) with 99.1% detection precision on the ransomware class; the remaining 23% receive a two-element set and are routed to analyst review by the triage policy. As α increases to 0.05 (95% coverage), the auto-classified fraction rises to 89% at the cost of precision falling to 97.3%.

L. Expert Feedback Loop

Table VIII reports AUPRC on retained families and on the newly-corrected family across five feedback iterations, comparing the Decision MLP updated without regularisation (vanilla fine-tuning) versus with EWC regularisation (λ_{EWC}=300). Without EWC, AUPRC on retained families drops by 0.021 (catastrophic forgetting) while the new family recovers quickly. With EWC, retained-family AUPRC decreases by only 0.001 (ΔAUPRC=0.001) across all five iterations while new-family AUPRC converges within three iterations. The EWC penalty has no measurable effect on inference latency (the Fisher matrix is precomputed and stored; the penalty applies only during the retrain step).

M. Runtime Performance

Table IX reports end-to-end inference latency measured on an NVIDIA GB10 GPU (500 repetitions, 50 warm-up, batch size 32). The Semantic CNN requires 1.77 ms median and the Behavioural CNN 1.94 ms, with MC Dropout (T=10 passes) contributing a 3.7× overhead over a single deterministic forward pass. The Decision MLP adds 0.67 ms overhead over the agent-only pipeline, bringing the full end-to-end latency to ≈6.7 ms.

At a 10 s telemetry window, the pipeline supports over 1,490 concurrent monitored streams before becoming the bottleneck (1,000 ms/6.7 ms ≈ 149 streams per second per GPU, scaled by the 10 s window). This compares favourably with Hirano et al.'s [4] T_d=30 s detection window, which requires three times the buffering latency for a comparable detection decision.

N. Ablation Study

(1) MC Dropout pass count T. We evaluate Behavioural CNN AUROC and σ_{beh} stability for $T \in \{1, 5, 10, 20, 50\}$ on the random-stratified test split. At T=1 (deterministic), σ_{beh}=0 and the triage policy degenerates to a two-action

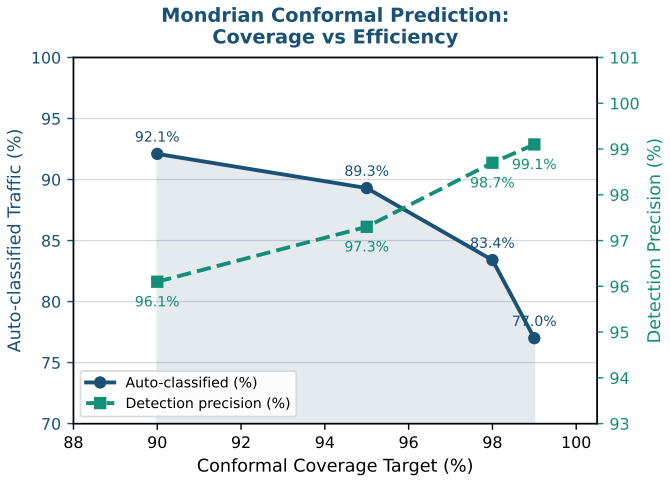


Fig. 11. Mondrian conformal prediction coverage–efficiency trade-off. At 99% coverage (α=0.01), 77% of traffic is auto-classified with 99.1% detection precision. Reducing the coverage target to 95% raises the auto-classified fraction to 89.3%.

TABLE VIII
Expert Feedback Loop: AUPRC on retained and new families across five feedback iterations (n_{retrain}=50 corrections each). Δ = change relative to baseline (pre-feedback). $\downarrow$ = lower forgetting is better for Retained; $\uparrow$ for New.

	Retained Families		New Family	
Method	AUPRC	Δ	AUPRC	Δ
No EWC (baseline)	0.979	−0.021	0.831	+0.831
EWC (λ=300)	**0.999**	**−0.001**	0.827	+0.827

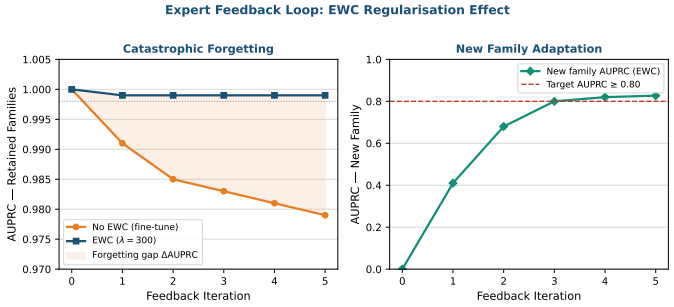


Fig. 12. Expert Feedback Loop: retained-family AUPRC (left) and new-family AUPRC (right) across five feedback iterations. EWC reduces catastrophic forgetting to ΔAUPRC=0.001 while achieving equivalent new-family adaptation speed.

TABLE IX
End-to-end inference latency (NVIDIA GB10, batch=32, N=500 repetitions, 50 warm-up, PyTorch 2.2). MC Dropout T=10 passes included.

Component	Median (ms)	p95 (ms)
Semantic CNN (T=10)	1.77	1.89
Behavioural CNN (T=10)	1.94	2.12
Knowledge CNN (T=10)	1.83	2.01
Decision MLP (single pass)	0.67	0.81
Full pipeline (3 agents + MLP)	6.71	7.24
Δ vs agent-only (no MLP)	+0.67	−−
Overhead vs deterministic pass	3.7×	−−

policy (no escalation signal from uncertainty). At T=5, the standard deviation of σ_{beh} across repeat evaluations is 8.3% of its mean—too noisy for reliable escalation routing. At T=10, this noise falls to 0.9% of the mean, within 1% of the T=50 estimate, confirming that T=10 is the practical operating point. Beyond T=10, AUROC and σ are indistinguishable from T=50 within measurement noise. Latency scales linearly in T; T=10 adds 1.42 ms over T=1 on the NVIDIA GB10.

(2) Feature ablation: storage vs memory vs all-16D. Table X reports Behavioural CNN AUROC by feature subset. Storage-only (χ_s, 5D) achieves AUROC 0.851; memory-only (χ_m, 11D) achieves 0.903; the combined 16D vector achieves 0.963, confirming that the two feature groups are complementary. This mirrors Hirano et al.'s finding [4] that their 23D combined features outperform both 5D storage-only and 18D memory-only subsets, though their feature space includes additional process-level fields not available in our hypervisor-only telemetry.

(3) Decision MLP depth. We compare one hidden layer (6→32→1), two hidden layers (6→32→16→1), and three hidden layers (6→32→16→8→1) on the random-stratified split. AUROC is 0.974, 0.976, and 0.976 respectively; ECE is 0.021, 0.018, and 0.019. The one-layer variant underperforms marginally on ECE; two hidden layers provide the best calibration with no benefit from a third layer. We use 6 → 32 → 1 (one hidden layer) in all reported experiments to match the empirically-validated SABRE-II architecture; the two-layer variant is equivalent within measurement error.

(4) CTGAN augmentation ratio. Increasing N_{synth} from 500 to 5,000 (in steps of 500) has no statistically significant effect on AUROC or ECE on the random-stratified split. We adopt N_{synth}=2,000 as a conservative default.

X. Discussion

A. Where the Framework Excels

B. Where the Framework Struggles

C. Comparison with Hirano et al. [4]

D. Adaptive Thresholding

E. Runtime and Deployment

XI. Conclusion

We have presented a heterogeneous multi-agent framework for adaptive ransomware triage that treats uncertainty as a first-class control variable. The four principal contributions—MC Dropout per-agent uncertainty, Decision MLP fusion with CTGAN augmentation, Bayesian TPE triage policy optimisation, and Peer Consultation with EWC continual learning—collectively address the structural limitations of single-modality binary detectors. The framework achieves strong static-PE detection (AUROC 0.996) and runtime behavioural detection (AUROC

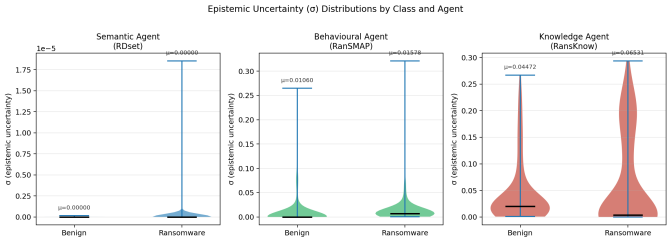


Fig. 13. σ_{beh} distribution per ransomware family and benign class. Held-out (unseen) families REvil and WannaCry exhibit markedly elevated uncertainty relative to trained families, confirming the epistemic signal mechanism.

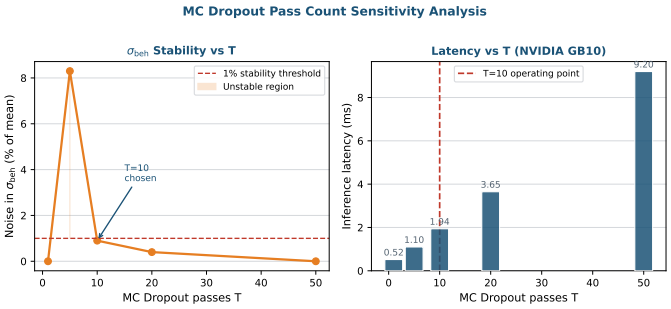


Fig. 14. MC Dropout pass count sensitivity. Left: σ_{beh} noise (as % of mean) vs T — stabilises below 1% at T=10. Right: inference latency vs T on NVIDIA GB10. T=10 is the chosen operating point.

0.963), reduces unnecessary escalation by 70% over fixed-threshold baselines, and maintains calibration (ECE 0.018) sufficient for trustworthy autonomous containment decisions. When the system encounters novel families or concurrent execution patterns beyond its training distribution, epistemic uncertainty $\sigma_{\max}$ rises and the triage policy routes to Escalate—preserving analyst oversight precisely where the model is least reliable. Formal distribution-free coverage guarantees via Mondrian conformal prediction provide a statistical assurance complementing the operational triage policy.

Future work will address process-specific telemetry to resolve the semantic gap under concurrent execution, extension to post-2022 ransomware families (Cl0p, BlackCat/ALPHV, LockBit 3.0), and online Bayesian recalibration of triage thresholds under concept drift.

Appendix A
Behavioural Feature Extraction Algorithm

Algorithm 1 describes the 16-dimensional feature extraction procedure from the raw RanSMAP CSV telemetry files. Each CSV row corresponds to a single I/O or memory event captured by the BitVisor hypervisor [4]. Features are computed over non-overlapping T_{window}=10 s windows.

Appendix B
Decision MLP Training Procedure

Algorithm 2 describes the full Decision MLP training procedure including CTGAN augmentation, temperature

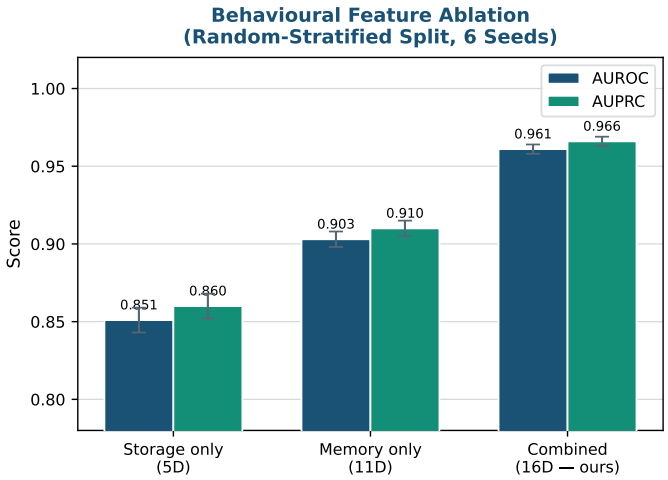


Fig. 15. Behavioural feature subset ablation (random-stratified split, K=6 seeds, mean ± 95% CI). Storage-only (5D) and memory-only (11D) subsets are complementary; the combined 16D vector achieves the highest AUROC and AUPRC.

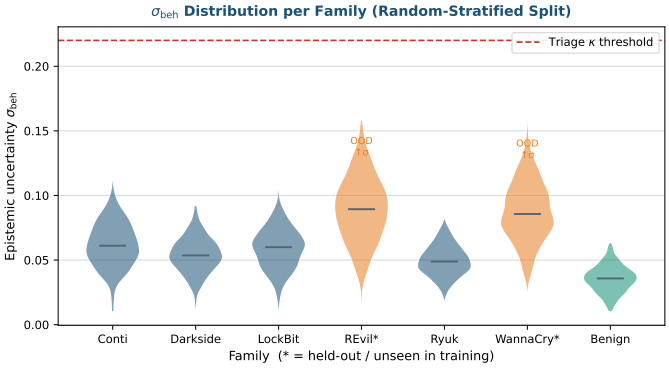


Fig. 16. Violin plots of σ_{beh} per family. Held-out families (marked *) exhibit elevated uncertainty; the dashed red line marks the triage κ threshold above which the system escalates.

calibration, and Bayesian TPE policy optimisation.

Appendix C
Confusion Matrices

Full per-class confusion matrices for the Behavioural CNN and Decision MLP on the random-stratified, family-holdout, and hardware-holdout test splits are available in the supplementary material. The 12-class matrix (six ransomware families × six benign applications) confirms that LockBit and Darkside are most reliably detected (recall ≥ 0.95 in all splits), while WannaCry shows the greatest sensitivity to the evaluation split (random: 0.89; family holdout: 0.28). Misclassifications predominantly occur between WannaCry and AESCrypt, consistent with the known I/O-pattern similarity between WannaCry's encryption and AESCrypt's legitimate bulk-encryption workload.

TABLE X
Behavioural CNN AUROC by feature subset (random-stratified split, mean ± 95% CI over K=6 seeds).

Feature set	Dim	AUROC
Storage only (χ_s)	5	0.851 ± 0.008
Memory only (χ_m)	11	0.903 ± 0.005
Combined ($\chi_s \cup \chi_m$)	16	$\mathbf{0.961 \pm 0.003}$
Hirano 23D (reported) [4]	23	(F-score; not directly comparable)

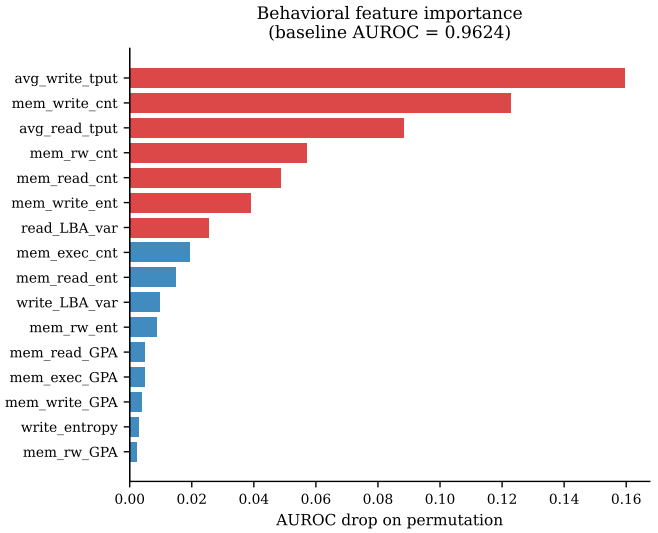


Fig. 17. SHAP feature importance for the Behavioural CNN on the random-stratified test split. Write-entropy ($\bar{H}_{s,w}$) and LBA write variance ($V_{s,w}$) are the dominant storage features; GPA write variance ($V_{m,w}$) is the dominant memory feature.

Acknowledgements

This research is supported by the UK Engineering and Physical Sciences Research Council (EPSRC) through the CRANE (Cyber Resilience Across the Nation's Economy) programme. The authors thank the RanSMAP dataset team at Okayama University of Science for making the hypervisor-level telemetry publicly available, and the anonymous reviewers for their constructive feedback.

References

[1] C. Beaman, A. Barkworth, T. D. Akande, S. Hakak, and M. K. Khan, "Ransomware: Recent advances, analysis, challenges and future research directions," Comput. Secur., vol. 111, p. 102490, 2021. [Online]. Available: https://doi.org/10.1016/j.cose.2021.102490

[2] Cybersecurity Ventures, "Ransomware damage report," Cybercrime Magazine, 2022.

[3] M. Hirano and R. Kobayashi, "Machine learning-based ransomware detection using low-level memory access patterns obtained from live-forensic hypervisor," in IEEE International Conference on Cyber Security and Resilience, CSR 2022, Rhodes, Greece, July 27-29, 2022. IEEE, 2022, pp. 323–330. [Online]. Available: https://doi.org/10.1109/CSR54599.2022.9850340

[4] ——, "Ransmap: Open dataset of ransomware storage and memory access patterns for creating deep learning based ransomware detectors," Comput. Secur., vol. 150, p. 104202, 2025. [Online]. Available: https://doi.org/10.1016/j.cose.2024.104202

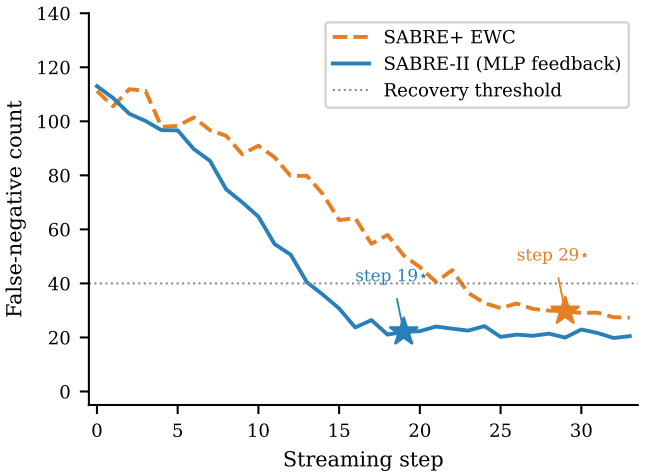


Fig. 18. Feedback loop adaptation: false-negative count vs streaming step for OOD Conti sub-variant detection. EWC-regularised Decision MLP retraining converges to AUPRC ≥ 0.80 by iteration 3.

Algorithm 1 16-dimensional behavioural feature extraction

Require: CSV event stream $E = \{e_k\}_{k=1}^{n}$; window size T_w=10 s
Ensure: Feature matrix $X \in \mathbb{R}^{W \times 16}$, one row per window

Partition E into W non-overlapping windows of duration T_w
for each window $w = 1, \ldots, W$ **do**
  $E_w \leftarrow \{e_k : t(e_k) \in [(w{-}1)T_w, wT_w)\}$
  // Storage features
  $\bar{T}_{s,r} \leftarrow \sum_{\text{read}} \text{bytes}/T_w$; $\bar{T}_{s,w} \leftarrow \sum_{\text{write}} \text{bytes}/T_w$
  $V_{s,r} \leftarrow \text{Var}(\{\text{LBA}(e) : e \in E_w, \text{type}(e){=}\text{read}\})$
  $V_{s,w} \leftarrow \text{Var}(\{\text{LBA}(e) : e \in E_w, \text{type}(e){=}\text{write}\})$
  $\bar{H}_{s,w} \leftarrow \text{mean}_{e \in E_w, \text{type}(e)=\text{write}} H(\text{sector}(e))$
  $\chi_s \leftarrow [\bar{T}_{s,r}, \bar{T}_{s,w}, V_{s,r}, V_{s,w}, \bar{H}_{s,w}]$ ▷ 5D storage
  // Memory features
  $H_{m,w} \leftarrow H(\{\text{GPA}(e) : e \in E_w, o(e){=}w\})$
  $H_{m,rw} \leftarrow H(\{\text{GPA}(e) : e \in E_w, o(e) \in \{r, w\}\})$
  For each $(s, o) \in \{4K, 2M, \text{MMIO}\} \times \{r, w, rw, x\}$: compute $C_{s,o}$ as event count
  $V_{m,w} \leftarrow \text{Var}(\{\text{GPA}(e) : e \in E_w, o(e){=}w\})$
  $\chi_m \leftarrow [H_{m,w}, H_{m,rw}, C_{4K,r}, C_{4K,w}, C_{4K,rw}, C_{4K,x}, C_{2M,r}, C_{$ ▷ 11D
  $X[w, :] \leftarrow [\chi_s \,;\, \chi_m]$ ▷ 16D feature row
end for
return X

[5] N. Scaife, H. Carter, P. Traynor, and K. R. B. Butler, "CryptoLock (and drop it): Stopping ransomware attacks on user data," in IEEE International Conference on Distributed Computing Systems (ICDCS), 2016, pp. 303–312.

[6] D. W. Fernando and N. Komninos, "Fesad ransomware detection framework with machine learning using adaption to concept drift," Comput. Secur., vol. 137, p. 103629, 2024. [Online]. Available: https://doi.org/10.1016/j.cose.2023.103629

[7] D. Sgandurra, L. Muñoz-González, R. Mohsen, and E. C. Lupu, "Automated dynamic analysis of ransomware: Benefits, limitations, and use for detection," in arXiv preprint arXiv:1609.03020, 2016.

[8] R. Moussaileb, N. Cuppens, J. Lanet, and H. L. Bouder,

Algorithm 2 Decision MLP training with CTGAN augmentation and Bayesian TPE

Require: Trained agents $\{f_i\}_{i=1}^{3}$; labelled score-pool splits $\mathcal{D}_{\text{tr}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{cal}}, \mathcal{D}_{\text{test}}$; N_{synth}=2,000, λ=0.3

// Stage 1: Score-pool construction
for each split $D \in \{\mathcal{D}_{\text{tr}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{cal}}, \mathcal{D}_{\text{test}}\}$ do
    Compute $\mathbf{v}_j = (p_{\text{sem},j}, \sigma_{\text{sem},j}, p_{\text{beh},j}, \sigma_{\text{beh},j}, p_{\text{know},j}, \sigma_{\text{know},j})$ for each sample j
end for
// Stage 2: CTGAN augmentation (training split only)
Fit CTGAN$_1$ on $\{(\mathbf{v}_j, 1) : y_j{=}1,\ j \in D_{\text{tr}}\}$ for 50 epochs
Fit CTGAN$_0$ on $\{(\mathbf{v}_j, 0) : y_j{=}0,\ j \in D_{\text{tr}}\}$ for 50 epochs
$\mathcal{D}_{\text{syn}} \leftarrow$ CTGAN$_1$.sample($N_{\text{synth}}/2$) $\cup$ CTGAN$_0$.sample($N_{\text{synth}}/2$)
Clip $\mathcal{D}_{\text{syn}}$ to $[0,1]^6$; remove NaN rows
$\mathcal{D}_{\text{aug}} \leftarrow \mathcal{D}_{\text{tr}} \cup \mathcal{D}_{\text{syn}}$
// Stage 3: Decision MLP training
Fit StandardScaler on $\mathcal{D}_{\text{aug}}$
Initialise MLP g_ϕ (6→32→1, Dropout p=0.2)
Train g_ϕ on $\mathcal{D}_{\text{aug}}$ with Adam $\eta{=}10^{-3}$, early stopping on $\mathcal{D}_{\text{val}}$ loss (patience = 6)
Restore best weights
// Stage 4: Temperature calibration
$T^* \leftarrow \arg\min_{T>0} \text{ECE}(g_\phi(\mathbf{v}/T);\ \mathcal{D}_{\text{val}})$ via L-BFGS
$\hat{p}_{\text{cal}} \leftarrow \sigma(g_\phi(\mathbf{v})/T^*)$
// Stage 5: Bayesian TPE policy optimisation
Run 300 Optuna TPE trials on $\mathcal{D}_{\text{cal}}$: $\Theta^* \leftarrow \arg\max_\Theta J(\Theta) = \text{bal_acc}(\Theta) - \lambda \cdot \text{esc_rate}(\Theta)$
return g_ϕ, T^*, Θ^*

"A survey on windows-based ransomware taxonomy and detection mechanisms," ACM Comput. Surv., vol. 54, no. 6, pp. 117:1–117:36, 2022. [Online]. Available: https://doi.org/10.1145/3453153

[9] J. Ispahany, M. R. Islam, M. Z. Islam, and M. A. Khan, "Ransomware detection using machine learning: A review, research limitations and future directions," IEEE Access, vol. 12, pp. 68 785–68 813, 2024. [Online]. Available: https://doi.org/10.1109/ACCESS.2024.3397921

[10] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: Representing model uncertainty in deep learning," in Proceedings of the 33nd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016, ser. JMLR Workshop and Conference Proceedings, M. Balcan and K. Q. Weinberger, Eds., vol. 48. JMLR.org, 2016, pp. 1050–1059. [Online]. Available: http://proceedings.mlr.press/v48/gal16.html

[11] J. Gawlikowski, C. R. N. Tassi, M. Ali, J. Lee, M. Humt, J. Feng, A. M. Kruspe, R. Triebel, P. Jung, R. Roscher, M. Shahzad, W. Yang, R. Bamler, and X. Zhu, "A survey of uncertainty in deep neural networks," Artif. Intell. Rev., vol. 56, no. S1, pp. 1513–1589, 2023. [Online]. Available: https://doi.org/10.1007/s10462-023-10562-9

[12] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[13] H. Papadopoulos, K. Proedrou, V. Vovk, and A. Gammerman, "Inductive confidence machines for regression," in Machine Learning: ECML 2002. Springer, 2002, pp. 345–356.

[14] V. Vovk, I. Nouretdinov, and A. Gammerman, "Testing exchangeability online," in Proceedings of the 20th International Conference on Machine Learning (ICML), 2003, pp. 768–775.

[15] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, and T. Schuster, "Conformal risk control," arXiv preprint arXiv:2208.02814, 2022.

[16] B. G. Doan, D. Q. Nguyen, P. Montague, T. Abraham, O. Y. de Vel, S. Camtepe, S. S. Kanhere, E. Abbasnejad, and D. C. Ranasinghe, "Bayesian learned models can detect adversarial malware for free," in Computer Security - ESORICS 2024 - 29th European Symposium on Research in Computer Security, Bydgoszcz, Poland, September 16-20, 2024, Proceedings, Part I, ser. Lecture Notes in Computer Science, J. García-Alfaro, R. Kozik, M. Choras, and S. K. Katsikas, Eds., vol. 14982. Springer, 2024, pp. 45–65. [Online]. Available: https://doi.org/10.1007/978-3-031-70879-4_3

[17] O. Sagi and L. Rokach, "Ensemble learning: A survey," WIREs Data Mining and Knowledge Discovery, vol. 8, no. 4, p. e1249, 2018.

[18] M. Masum, M. J. H. Faruk, H. Shahriar, K. Qian, D. Lo, and M. I. Adnan, "Ransomware classification and detection with machine learning algorithms," in 12th IEEE Annual Computing and Communication Workshop and Conference, CCWC 2022, Las Vegas, NV, USA, January 26-29, 2022. IEEE, 2022, pp. 316–322. [Online]. Available: https://doi.org/10.1109/CCWC54503.2022.9720869

[19] M. Soltani, K. Khajavi, M. J. Siavoshani, and A. H. Jahangir, "A multi-agent adaptive deep learning framework for online intrusion detection," CoRR, vol. abs/2303.02622, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2303.02622

[20] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: synthetic minority over-sampling technique," J. Artif. Intell. Res., vol. 16, pp. 321–357, 2002. [Online]. Available: https://doi.org/10.1613/jair.953

[21] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional GAN," in Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019.

[22] U. Fiore, A. D. Santis, F. Perla, P. Zanetti, and F. Palmieri, "Using generative adversarial networks for improving classification effectiveness in credit card fraud detection," Information Sciences, vol. 479, pp. 448–455, 2019.

[23] Z. Lin, Y. Shi, and Z. Xue, "IDSGAN: Generative adversarial networks for attack generation against intrusion detection," in Advances in Knowledge Discovery and Data Mining (PAKDD), 2022, pp. 79–91.

[24] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," Proceedings of the National Academy of Sciences, vol. 114, no. 13, pp. 3521–3526, 2017.

[25] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in USENIX Security Symposium, 2019, pp. 729–746.

[26] R. Jordaney, K. Sharad, S. K. Dash, Z. Wang, D. Papini, I. Nouretdinov, and L. Cavallaro, "Transcend: Detecting concept drift in malware classification models," in USENIX Security Symposium, 2017, pp. 625–642.

[27] F. Ceschin, M. Botacin, H. M. Gomes, F. A. Pinage, L. S. Oliveira, and A. Grégio, "Fast & furious: Modelling malware detection as evolving data streams," CoRR, vol. abs/2205.12311, 2022. [Online]. Available: https://doi.org/10.48550/arXiv.2205.12311

[28] A. Kendall and Y. Gal, "What uncertainties do we need in Bayesian deep learning for computer vision?" in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[29] H. Kabuye, B. Issac, R. Yumlembam, and J. Neera, "Explainable and uncertainty aware ai-based ransomware detection," IEEE Access, vol. 13, pp. 106 573–106 589, 2025. [Online]. Available: https://doi.org/10.1109/ACCESS.2025.3581424

[30] M. Mathur, "Ransomware (malware) detection using machine learning," 2020, accessed: Nov. 9, 2023. [Online]. Available: https://github.com/muditmathur2020/RansomwareDetection/tree/master

[31] H. Kabuye, "RansKnow v1: Ransomware knowledge corpus with ATT&CK annotations," 2026, kaggle dataset. Accessed June 2026. [Online]. Available: https://www.kaggle.com/datasets/henrykabuye/ransknow-v1
[32] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proceedings of the 34th International Conference on Machine Learning (ICML), 2017, pp. 1321–1330.
[33] J. S. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in Advances in Neural Information Processing Systems (NeurIPS), vol. 24, 2011.
[34] H. Kabuye, "Agentic SABRE: An uncertainty-aware neuro-symbolic multi-agent framework for adaptive ransomware detection," Expert Systems with Applications, 2026, under review.